RT-FedMTL: A Novel Real-Time Federated Multi-Task Learning Framework with Small Language Models for Natural Language Understanding Tasks

Quang-Vinh Pham and Quang-Hung Le

***Abstract*—Natural language understanding is a crucial component in intelligent systems such as virtual assistants, conversational agents, and information retrieval. Deploying these systems in real application scenarios requires not only accurate but also efficient, privacy-preserving, and capable of handling multiple tasks simultaneously. Centralized training of large language models based approaches raises significant challenges in terms of data privacy, communication overhead, and real-time inference latency. This paper proposes RT-FedMTL, a novel real-time federated multi-task learning framework with small language models for natural language understanding tasks. Particularly, our framework integrates federated learning and multi-task learning to enable collaborative training across distributed clients while preserving data locality and leveraging shared representations across tasks. We utilize WebSocket protocols and small language models tuning to reduce the feedback loop and provide near-instantaneous global updates. Extensive experiments conducted on five small language model architectures with respect to three standard natural language understanding tasks to get the best trade-off between performance and resource usage. A significant result demonstrates RT-FedMTL can drastically minimize per-client GPU memory requirements (up to 74-93%) versus centralized multi-task learning baselines, while keeping performance for multiple benchmarks. The source code is available at: https://github.com/vinh1988/RT-FedMTL.git.**

***Index Terms*—Federated Learning, Multi-Task Learning, Federated Multi-Task Learning, Small Language Models, Natural Language Understanding**

I. Introduction

Natural language understanding (NLU) is a core module in advanced artificial intelligence language processing systems such as sentiment analysis [1] for customer feedback, intent classification [2] for conversational agents, semantic similarity [3] for information retrieval, and so on. According to the modern approach, these sytems have achieved impressive capabilities by leveraging large language models [4]. However, they often require centralized training and deployment, which poses challenges related to data privacy, computational cost, and inference latency.

Consequently, recent research has shifted towards small language models (SLMs) [5], [6], [7] due to their low inference latency and cost-effectiveness. These models are well-suited to resource-constrained environments, addressing the challenges of large language models [8]. Nevertheless, SLMs are often limited in capacity, making it difficult to maintain high performance across a variety of NLU tasks.

At the same time, multi-task learning (MTL) [9], [10] has been widely adopted as an effective paradigm to solve multiple tasks simultaneously. This approach of leveraging useful information from related tasks can lead to better learning performance [2], [11]. Despite its advantages, most existing multi-tasking learning methods typically require centralized training data, which is impractical in privacy-sensitive situations where data is distributed across multiple clients.

With the growing availability of sensitive textual data, strict privacy regulations such as the GDPR [12] have raised concerns about centralizing user data in conventional machine learning pipelines. These challenges have motivated the development of federated learning (FL), a distributed paradigm that enables model training across clients without transferring raw data to a central server, thereby preserving privacy while supporting large-scale learning [13], [14]. However, current FL pipelines mainly depend on batch synchronization where clients execute multiple local steps and will execute the sync periodically [13], [15]. This design is effective for static datasets but less so for streaming and dynamic environments where language patterns as well as subjects and user intent change with frequency.

To address the aforementioned challenges and concerns, this paper proposes RT-FedMTL, a novel real-time federated MTL framework with SLMs for NLU

Quang-Vinh Pham is with the Department of Data Science, TMA Tech Group, Gia Lai, Vietnam (e-mail: pqvinh@tma.com.vn).

Quang-Hung Le is with the Department of Information Technology, Quy Nhon University, Vietnam (e-mail: lequanghung@qnu.edu.vn).

* Corresponding author: Quang-Hung Le (lequanghung@qnu.edu.vn)

tasks. Particularly, our framework integrates FL and MTL to enable collaborative training across distributed clients while preserving data locality and leveraging shared representations across tasks. To the best of our knowledge, this is the first work to explore the use of WebSocket protocols and SLMs tuning to reduce the feedback loop and provide near-instantaneous global updates for a NLU-based federated MTL system. Our key contributions are summarized as follows:

1) We propose RT-FedMTL, a novel real-time federated framework of MTL and SLMs to harness cross-task knowledge transfer without causing a large amount of resources to be consumed on devices.
2) We assess five SLM architectures (DistilBERT [5], BERT-Medium [6], BERT-Mini [6], MiniLM [7], and TinyBERT [16]) with respect to three standard NLU tasks (sentiment analysis, paraphrase detection, and semantic similarity), to get the best trade-off between performance and resource usage.
3) We investigate the effects of non-IID and task heterogeneity on convergence speed and task accuracy in real-time domain.
4) We show RT-FedMTL can drastically minimize per-client GPU memory requirements (up to 74-93%) versus centralized MTL baselines, while keeping performance for multiple benchmarks (GLUE suite).

The remainder of the paper is organised as follows. Section II defines the formal problem of RT-FedMTL and Section III presents the proposed framework. Section IV describes the experimental setup and results analysis. The relevant work is briefly reviewed in Section V. Finally, Section VI concludes and outlines future work.

II. PROBLEM DEFINITION

This section introduces the official RT-FedMTL environment for NLU. We define the formal mathematical structure for distributed multi-task optimization work, tackle the issues of data and system heterogeneity, and set the global goal for training collaboration models across private edge clients.

A. RT-FedMTL Formalization for NLU

We model a NLU-based RT-FedMTL system in which training is carried out across communication rounds $t = 1, 2, \ldots$ [13], [17], [18]. In each round, a subset of clients is chosen, local updates are made to private data streams, and parameters are returned to a central server for aggregation. As opposed to offline FL, the client data distributions can vary over time. Hence, both local goals and optimum parameters are time-dependent. Let $\mathcal{C} = \{1, \ldots, C\}$ represent the set of clients and $\mathcal{K} = \{1, \ldots, K\}$ represent the set of NLU tasks. Each client $c \in \mathcal{C}$ is assigned to only one main task $\tau(c) \in \mathcal{K}$ (or a small subset of tasks), where new samples are added to the local stream perpetually. The goal is to learn a federated model with cross-task transfer quality but task-level specialization in the absence of IID and time-varying data.

B. Distribution of Clients and the Data

Heterogeneity comes in two forms:

- Heterogeneity of Data: the local distributions will differ across clients c_i, c_j (i.e. $P_{c_i}(x, y) \neq P_{c_j}(x, y)$) (label skew, domain shift, vocabulary differences).
- Task heterogeneity: clients might optimize different losses according to $\tau(c)$, which will produce diverging gradients across tasks.

In addition, systems remain heterogeneous, stale updates are propagated by client availability and update frequency. Therefore, the optimization problem must include partial participation and nonuniform local progress. Let $\ell_{\tau(c)}(\theta_e, \theta_{\tau(c)}; \xi)$ be the loss cost for the specific sample $\xi \in \mathcal{D}_c^t$. The universal goal at stage t is

$$\min_{\theta_e, \{\theta_k\}} F_t(\Theta) = \sum_{c=1}^{C} p_c^t \, \mathbb{E}_{\xi \sim \mathcal{D}_c^t} \left[\ell_{\tau(c)}(\theta_e, \theta_{\tau(c)}; \xi) \right] + \lambda \, \Omega(\{\theta_k\}), \tag{1}$$

where p_c^t denote the weight of the aggregated model (i.e., the weight per unit local sample), Ω is a regularizer to enhance the cross-task learning stability and $\lambda \geq 0$ moderates the sharing-specialization tradeoff. After local updates, the server also aggregates common encoder parameters through weighted averaging,

$$\theta_e^{t+1} = \sum_{c \in \mathcal{C}_t} \alpha_c^t \, \theta_{e,c}^{t+1}. \tag{2}$$

and updates custom task-specific parameters, following task-wise aggregation,

$$\theta_k^{t+1} = \sum_{c \in \mathcal{C}_t \colon \tau(c) = k} \beta_c^t \, \theta_{k,c}^{t+1}. \tag{3}$$

In $\mathcal{C}_t$ the client subset participates at round t and α_c^t, β_c^t can be viewed as normalized aggregation coefficients [13], [17], [19]. By working towards privacy and heterogeneity requirements, RT-FedMTL is captured as the architecture in real-time, jointly maximizing the optimal use of shared representations and task-specific parameters (3, 4).

III. PROPOSED METHOD

The proposed RT-FedMTL confronts the problems simultaneously when balancing privacy-preserving NLU and resource starved edge computing. We propose a three-integrated architecture of a hierarchical federated-based architecture tailored for real-time asynchronous synchronization across WebSockets, multi-task learning, based on common linguistic representations, but for task-specific specialization, and parameter-effective SLMs to reduce computing and communication footprint on edge devices.

A. Overview of the System

This paper explores an example of a multistructure federated multi-task framework for real-time NLU applications. As illustrated in Fig. 1, a high level orchestration server, and several heterogeneous client nodes form the basis of the architecture. A parameter server maintains a global state at the center and controls the model aggregation logic. The communications depend on low-latency WebSockets for a bidirectional exchange of model updates, and aggregated parameters.

Federated components and privacy constraints of a system are built on:

- Global server $\mathcal{S}$: orchestrates client sampling, model broadcast, and aggregation.
- Distributed clients $\mathcal{C}$: edge devices/silos to store the raw text.
- Task Set $\mathcal{K}$: the heterogeneous NLU tasks (intent classification, sentiment analysis, sequence labeling).
- Local Data sets $\mathcal{D}_c^t$: one private time-stamped data set for the client c at round t that is private.

For the sake of privacy, samples are never uploaded. Each of these clients sends only model updates (or compressed adapter and head updates) and acts as host-side aggregation [13], [20]. This is in line with the traditional federated privacy model in which data locality is retained and collaborative training is advocated in the field.

B. Backbone of small language model

This approach enables special SLMs for ensuring an optimal computational efficiency with high language understanding. We use architectures such as DistilBERT [5], BERT-Medium [6], MiniLM [7] for resource-constrained environments. The SLM backbone consists of several integral stages:

a) Tokenizer and an Embedding Layer: Outputs are inputted and processed by input text subword tokenizer that converts raw strings into a series of tokens. The generated tokens map to a high-dimensional vector space by means of a learned embedding layer. To model sequence structures without the overhead of recurrences, we add learnable position embeddings at the level of the token embeddings providing the machine with necessary context about the order of the tokens.

b) Low-Cost Transformer Encoder: The central element is a lightweight transformer encoder. These models consist of fewer layers and smaller hidden dimensionality than traditional BERT-base architectures. Each layer utilizes multi-head self-attention for long-range dependencies and the feed-forward network for feature transformation. This streamlined way significantly reduces parameter number and computational load, thus allowing for fast inference and training on edge devices.

c) Shared representation: The encoder generates a dense vector, represented as a shared representation $f_e(x;\theta_e)$. This captures the basic semantically and syntactically fundamental information in the input. And in our multi-task framework, though, with the backbone shared across all tasks the model learns universal linguistic patterns while task specific heads focus on unique output. This federated learning-based collaborative tuning guarantees strong generalization across heterogeneous client data.

C. Strategies for Multi-Task Learning

We adopt a hybrid approach of sharing the parameters. Throughout the tasks that all share linguistic representation, the lower (encoder) layers of the transformer remains unchanged. On the other hand, top layers are task-centric classification or regression heads respectively. We thus maintain an adaptive loss weighting mechanism to minimize negative transfer:

$$\mathcal{L}_{total} = \sum_{j=1}^{T} \alpha_j \mathcal{L}_j \tag{4}$$

where α_j gets dynamically tuned for each task j depending on the convergence rate.

We define the parameters

$$\Theta = \{\theta_e, \{\theta_k\}_{k=1}^{K}\}, \tag{5}$$

where θ_e is a well-known small-language-model encoder and θ_k is the task-based prediction head of task k [18], [19], [20]. Prediction is for input-output joint (x, y) set task k.

$$\hat{y} = h_k\big(f_e(x;\theta_e);\theta_k\big), \tag{6}$$

where $f_e(\cdot)$ is the shared encoder and $h_k(\cdot)$ is the task head. It encourages sharing of parameters among easily transferable linguistic traits but preserves task-specificity.

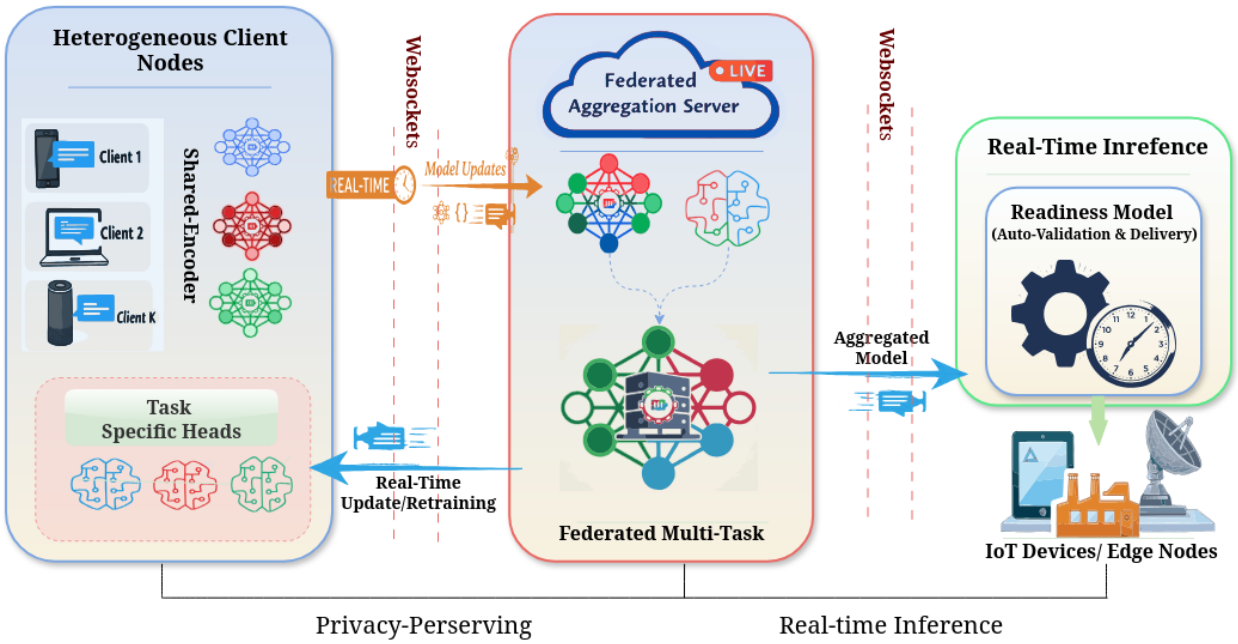

Fig. 1: Architecture of the Developed Frameworks for Real-time Federated Multi-Task Learning. The system allows seamless integration of streaming data processing with collaborative model refinement.

D. Federated Optimization and Real-Time Adaptation

The federated optimization process is described in Algorithm 1. This system enables real-time adaptation by dynamically synchronizing updates on request via WebSockets. Using a task-aware version of FedAvg [13], the server aggregates updates and normalizes noise from heterogeneous sources (non-IID). Most recent work in adaptive federated optimization [21] suggests incorporating momentum and adaptive learning rates will add an even greater level of stability.

E. Communication-Efficient Fine-Tuning (LoRA)

To further reduce resource usage in edge devices, we perform LoRA [22]. This ranges from updating in local B, where instead of updating the full weight matrices $W \in \mathbb{R}^{d\times d}$, we optimize the low rank decomposition matrices A and B, where:

$$W' = W + \Delta W = W + BA. \tag{7}$$

$A \in \mathbb{R}^{r\times d}$ and $B \in \mathbb{R}^{d\times r}$ with rank $r \ll d$. When performing big updates, only the matrices A and B are sent, minimizing communication volume up to 99

IV. EXPERIMENTS

In this section, the proposed RT-FedMTL framework is examined in terms of five thematic dimensions: overall baseline comparisons, the impact of MTL, paradigm performance gaps, real-time suitability, and communication efficiency.

Algorithm 1 RT-FedMTL: Real-Time Federated Multi-Task Learning

Server Initialize: Shared core ϕ_0, task heads $\{\psi_{j,0}\}_{j=1}^{T}$
for each round $t = 1, 2, \ldots$ **do**
  Server sends $\phi_t, \psi_{Task(k),t}$ to client $k \in K$ via WebSockets
  for each client $k \in K$ in parallel **do**
    Local training: $\{\Delta\phi_k, \Delta\psi_{Task(k),k}\} \leftarrow$ LocalUpdate$(\mathcal{D}_k, \phi_t, \psi_{Task(k),t})$
    Updates $\{\Delta\phi_k, \Delta\psi_{Task(k),k}\}$ via WebSockets to Server
  end for
  Task-Aware Aggregation:
    $\phi_{t+1} \leftarrow \phi_t + \eta \sum_{k=1}^{K} \frac{n_k}{N} \Delta\phi_k$ {Universal Knowledge}
  for each task $j \in \{1, \ldots, T\}$ in parallel **do**
    $\psi_{j,t+1} \leftarrow \psi_{j,t} + \eta \sum_{k\in K_j} \frac{n_k}{N_j} \Delta\psi_{j,k}$ {Heads Specialization}
  end for
  Server assigns $\{\phi_{t+1}, \psi_{Task(k),t+1}\}$ for Local Validation
end for

A. Experimental Setup

1) Datasets and Tasks: This evaluation combines three core NLU benchmarks that represent different NLU problems from within the GLUE benchmark [2]. SST-2

TABLE I: Experimental Design Matrix

Exp.	Paradigm	Task Config.	Clients	Data Dist.
1	Centralized MTL	Multi-Task	3	Full access
2	Centralized Single	SST-2 only	1	Full access
3	Centralized Single	QQP only	1	Full access
4	Centralized Single	STS-B only	1	Full access
5	Federated MTL	Multi-Task	3	IID
6	Federated MTL	Multi-Task	3	IID + LoRA
7	Federated MTL	Multi-Task	9	Non-IID (3×3)
8	Federated Single	QQP only	3	Non-IID
9	Federated Single	SST-2 only	3	Non-IID
10	Federated Single	STS-B only	3	Non-IID

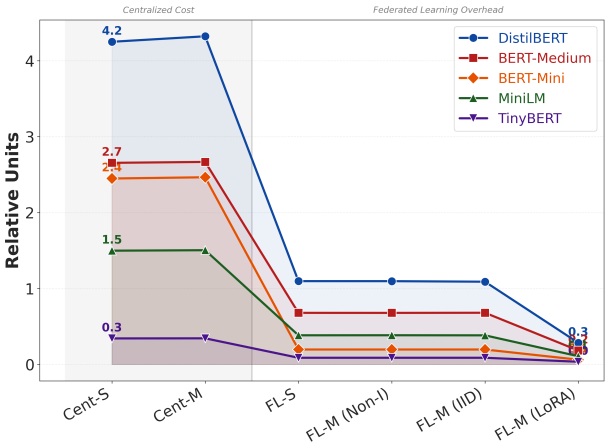

Fig. 2: Resource Usage Trajectory. Federated settings produce a clear 3-4× reduction in per-client resource usage relative to centralized training because each client holds only a data partition. LoRA further halves resource demands.

[1] is a binary classification task (66,477 training, 872 validation, 1,821 test samples) for detecting sentiment polarity. The performance is determined by accuracy and F1-score [23]. QQP [24] is a binary classification task for paraphrase detection with 323,415 training, 40,431 validation, and 39,081 test samples. Evaluation uses accuracy and F1-score [23]. STS-B [3] is a regression task that scores the semantic similarity of sentence pairs on a 1 to 5 scale, with 4,249 training, 1,500 validation, and 1,379 test samples. Performance is evaluated by Pearson [25] and Spearman correlation [26].

subsubsectionFederated Data Partitioning The data is uniformly distributed among clients in IID Partitioning (Experiments 5-6 in Table I, 3 clients, multi-task scenarios). For Non-IID Partitioning the Dirichlet distribution (α=0.5), which simulates real heterogeneity of data, uses three clients per task (9 total), giving 107,805 / 22,159 / 1,416 samples per client for QQP, SST-2, and STS-B respectively (Experiments 7-10 in Table I). Validation sets are *not* split over clients in any paradigm, which guarantees global model assessment consistency.

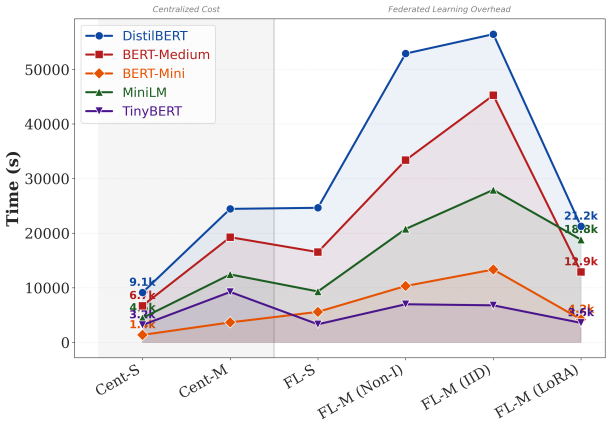

Fig. 3: Training Time Trajectory capturing the wall-clock training durations. The transition from centralized to federated multi-task settings introduces a significant overhead, which LoRA substantially offsets by limiting parameter transmission.

2) Baselines: Three baselines are tested: (1) Centralized MTL: all 5 SLM types trained on centralized data and share encoder and task head, this is used as the top-tier reference (Exp. 1 in Table I); (2) Federated Single-Task: per-task federated experiments with 3 clients and non-IID data (Exps. 8-10 in Table I); and (3) Large-model federated methods: evaluation against larger LMs to see communication efficiency and performance trade-offs.

3) Evaluation Metrics: Task-level metrics are accuracy & F1-score [23] (SST-2, QQP) and Pearson [25] / Spearman correlation [26] (STS-B) [2]. Federated system-level metrics are communication cost per round, convergence speed, and total training time.

4) Implementation Details: All experiments use learning rate 2×10^{-5} with 1,000-step warmup, batch size 8 for all experiments (Centralized to FL/FL-MTL), weight decay 0.01, gradient clipping at the max-norm of 1.0, and random seed 42. Centralized are trained with 25 epochs; federated clients train 1 local epoch per round. Communication is provided by ports 8771-8775 with 60 s client timeout and 3,400 s round timeout and 3-retry exponential-backoff.

B. Results and Discussion

1) Overall Performance (Comparison across Baselines): . Table IV shows that small language models (SLMs) within RT-FedMTL still achieve good performance in NLU. Centralized MTL is at least a baseline; however, DistilBERT in federated MTL (IID) performance, surprisingly, is 0.9461 on SST-2, better than that of its centralized counterpart. MiniLM features a very low resource-to-function ratio with 0.8966 and 0.8384 in

TABLE II: Resource Usage Trajectory (GB, per-client average)

Model	Cent-S	Cent-M	FL-S	FL-M (Non-IID)	FL-M (IID)	FL-M (LoRA)
DistilBERT	4.2492	4.3214	1.0972	1.0968	1.0896	0.2852
BERT-Medium	2.6549	2.6671	0.6795	0.6790	0.6807	0.1840
BERT-Mini	2.4487	2.4651	0.1969	0.1969	0.1969	0.0630
MiniLM	1.4982	1.5045	0.3839	0.3846	0.3835	0.1090
TinyBERT	0.3427	0.3440	0.0881	0.0881	0.0881	0.0356

Cent-S: Centralized Single-Task, Cent-M: Centralized Multi-Task, FL-S: Federated Single-Task, FL-M: Federated Multi-Task, IID/Non-IID: Data Distribution, LoRA: Low-Rank Adaptation.

TABLE III: Training Time Trajectory (seconds)

Model	Cent-S	Cent-M	FL-S	FL-M (Non-IID)	FL-M (IID)	FL-M (LoRA)
DistilBERT	9,122	24,450	24,648	52,904	56,454	21,245
BERT-Medium	6,693	19,257	16,527	33,375	45,258	12,886
BERT-Mini	1,359	3,653	5,581	10,318	13,341	4,192
MiniLM	4,473	12,437	9,317	20,766	27,930	18,808
TinyBERT	3,170	9,228	3,308	6,975	6,778	3,561

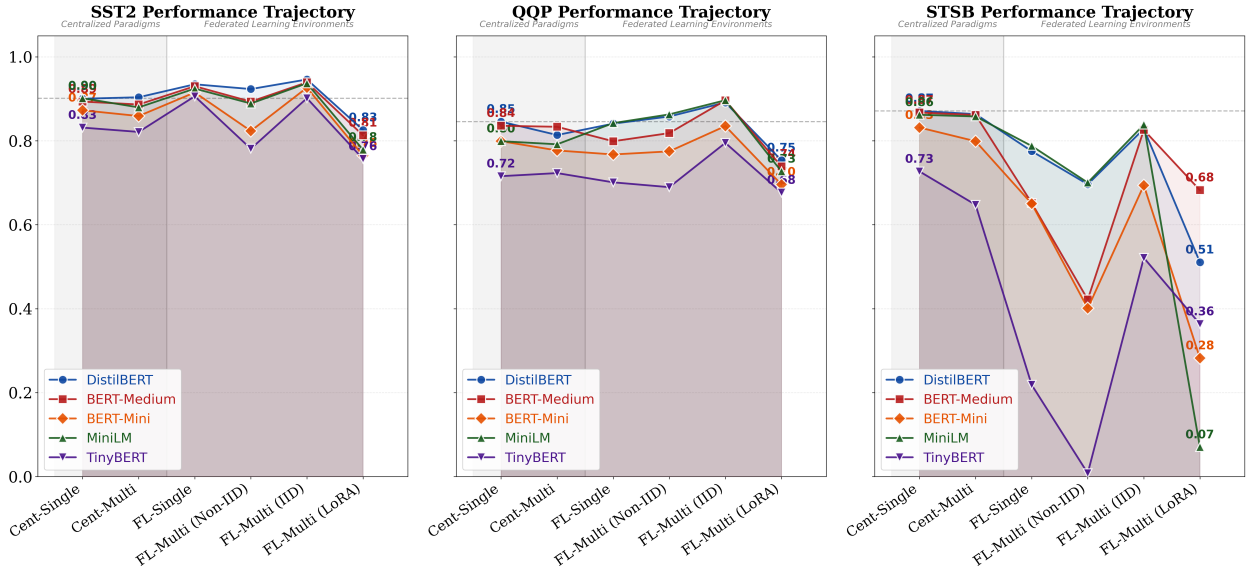

Fig. 4: Performance across six paradigm-complexity stages for five SLM architectures. Dashed lines separate centralized and federated settings.

QQP and STS-B tasks using IID respectively-scores that are on par or better than those of much larger models and taking only 0.38 GB of GPU memory (compared with 1.09 GB in DistilBERT). It confirms that in distributed NLU the size of the model does not only matter for the quality, and compact architectures such as MiniLM can be fine-tuned with respect to it.

2) Effect of Multi-Task Learning: MTL offers notable benefits in federated scenarios on account of its implementation of shared representations across similar tasks. As evidenced in Figure 4 and Table IV, it is evident that comparing FL-Single and FL-MTL (Non-IID) has rich task interplay. SST-2 shows mild negative transfer in some setups, whereas it is usually worse for regression problems such as STS-B for Non-IID situations. Nevertheless, classification tasks tend to demonstrate more positive transfer, with features learned by SST-2 benefiting QQP for example. Large SLMs such as DistilBERT and BERT-Medium exhibit robustness to the negative transfer, because they have a higher representational ability.

3) Federated Learning Performance: A "Reliability Gap" emerges as the switching from centralized to federated learning is observed from a moderate quality ($\leq 5\%$) degradation in classification for tasks such as SST-2 and QQP. This difference is most prominent in the STS-B Pearson correlation, which decreases notably for smallest architectures such as TinyBERT under the non-IID MTL

TABLE IV: Comparative Analysis of SLM Performance across Paradigms

					SST-2		QQP		STS-B		System	System
Model	Paradigm	Task	Dist.	Tuning	Acc.	F1	Acc.	F1	Pear.	Spea.	Time (s)	Res (GB)
DistilBERT [5]	FL	ST	Non-IID	FFT	—	—	0.8408	0.7724	—	—	34904.44	1.0956
	FL	ST	Non-IID	FFT	—	—	—	—	0.7753	0.7421	15452.52	1.0988
	FL	ST	Non-IID	FFT	0.9346	0.9392	—	—	—	—	23586.44	1.0972
	FL	MTL (ours)	IID	LoRA	0.8257	0.8433	0.7535	0.6754	0.5108	0.4919	21245.26	0.2852
	FL	MTL (ours)	Non-IID	FFT	0.9232	0.9298	0.8579	0.8054	0.6963	0.6967	52903.97	1.0968
	FL	MTL (ours)	IID	FFT	0.9461	0.9514	0.8913	0.8610	0.8284	0.8136	56454.46	1.0896
	Cent.	ST	IID	FFT	—	—	—	—	0.8712	0.8691	4561.78	4.2470
	Cent.	ST	IID	FFT	0.9002	0.9061	—	—	—	—	18514.82	4.2460
	Cent.	ST	IID	FFT	—	—	0.8456	0.8934	—	—	4290.59	4.2547
	Cent.	MTL	IID	FFT	0.9037	0.9077	0.8137	0.8778	0.8635	0.8614	24449.68	4.3214
BERT-Medium [6]	FL	ST	Non-IID	FFT	—	—	0.7989	0.6852	—	—	23880.89	0.6789
	FL	ST	Non-IID	FFT	—	—	—	—	0.6521	0.6212	10261.92	0.6810
	FL	ST	Non-IID	FFT	0.9300	0.9348	—	—	—	—	15439.04	0.6786
	FL	MTL (ours)	Non-IID	FFT	0.8933	0.8988	0.8183	0.7441	0.4219	0.4132	33374.50	0.6790
	FL	MTL (ours)	IID	FFT	0.9392	0.9450	0.8962	0.8655	0.8258	0.8052	45258.21	0.6807
	FL	MTL (ours)	IID	LoRA	0.8131	0.8362	0.7388	0.6805	0.6823	0.6512	12885.74	0.1840
	Cent.	ST	IID	FFT	—	—	—	—	0.8673	0.8648	3381.83	2.6467
	Cent.	ST	IID	FFT	0.8933	0.8970	—	—	—	—	13492.62	2.6711
	Cent.	ST	IID	FFT	—	—	0.8358	0.8859	—	—	3205.51	2.6467
	Cent.	MTL	IID	FFT	0.8865	0.8884	0.8333	0.8777	0.8615	0.8584	19257.40	2.6671
BERT-Mini [6]	FL	ST	Non-IID	FFT	—	—	0.7674	0.6201	—	—	9642.59	0.1969
	FL	ST	Non-IID	FFT	—	—	—	—	0.6504	0.6150	2547.10	0.1969
	FL	ST	Non-IID	FFT	0.9151	0.9196	—	—	—	—	4552.75	0.1969
	FL	MTL (ours)	Non-IID	FFT	0.8234	0.8226	0.7748	0.6668	0.4009	0.3939	10317.55	0.1969
	FL	MTL (ours)	IID	LoRA	0.7649	0.7915	0.6958	0.5602	0.2824	0.2773	4191.67	0.0630
	FL	MTL (ours)	IID	FFT	0.9266	0.9329	0.8350	0.7981	0.6940	0.6550	13341.01	0.1969
	Cent.	ST	IID	FFT	—	—	—	—	0.8316	0.8283	411.45	2.4487
	Cent.	ST	IID	FFT	0.8727	0.8773	—	—	—	—	3348.26	2.4487
	Cent.	ST	IID	FFT	—	—	0.7990	0.8536	—	—	318.61	2.4487
	Cent.	MTL	IID	FFT	0.8589	0.8650	0.7770	0.8580	0.7989	0.7973	3652.50	2.4651
MiniLM [7]	FL	ST	Non-IID	FFT	—	—	0.8421	0.7644	—	—	14583.05	0.3842
	FL	ST	Non-IID	FFT	—	—	—	—	0.7877	0.7489	4920.48	0.3840
	FL	ST	Non-IID	FFT	0.9243	0.9293	—	—	—	—	8447.20	0.3836
	FL	MTL (ours)	IID	FFT	0.9369	0.9428	0.8966	0.8662	0.8384	0.8197	27930.06	0.3835
	FL	MTL (ours)	Non-IID	FFT	0.8888	0.8974	0.8629	0.8150	0.7003	0.7061	20765.57	0.3846
	FL	MTL (ours)	IID	LoRA	0.7787	0.8102	0.7271	0.6060	0.0703	0.0621	18808.08	0.1090
	Cent.	ST	IID	FFT	—	—	—	—	0.8620	0.8592	2696.27	1.4929
	Cent.	ST	IID	FFT	0.9014	0.9051	—	—	—	—	8334.10	1.4950
	Cent.	ST	IID	FFT	—	—	0.7990	0.8656	—	—	2388.13	1.5067
	Cent.	MTL	IID	FFT	0.8796	0.8829	0.7917	0.8649	0.8580	0.8560	12437.12	1.5045
TinyBERT [16]	FL	ST	Non-IID	FFT	—	—	0.7010	0.3887	—	—	6226.63	0.0881
	FL	ST	Non-IID	FFT	—	—	—	—	0.2186	0.1755	1182.03	0.0881
	FL	ST	Non-IID	FFT	0.9060	0.9122	—	—	—	—	2514.06	0.0881
	FL	MTL (ours)	IID	FFT	0.9014	0.9145	0.7950	0.7494	0.5212	0.4754	6778.00	0.0881
	FL	MTL (ours)	Non-IID	FFT	0.7810	0.7782	0.6895	0.4733	0.0081	0.0063	6975.28	0.0881
	FL	MTL (ours)	IID	LoRA	0.7569	0.7902	0.6766	0.4475	0.3636	0.3505	3560.68	0.0356
	Cent.	ST	IID	FFT	—	—	—	—	0.7270	0.7230	2337.42	0.3427
	Cent.	ST	IID	FFT	0.8314	0.8358	—	—	—	—	4857.08	0.3427
	Cent.	ST	IID	FFT	—	—	0.7157	0.8139	—	—	2315.39	0.3427
	Cent.	MTL	IID	FFT	0.8211	0.8263	0.7230	0.8101	0.6474	0.6426	9227.55	0.3440

ST = Single-Task, MTL = Multi-Task Learning, Cent. = Centralized, LoRA = Low-Rank Adaptation, FFT = Full Fine-Tuning, Pear. = Pearson Correlation, Spea. = Spearman Correlation.

scenario. This line is illustrated in Figure 4, which illustrates how a proposed RT-FedMTL does in fact mitigate this gap by stabilizing performance without loss of signal across tasks. Notably, DistilBERT is generally the most stable architecture for sentiment-heavy tasks, as it shows only a slow decay performance even under the worst of non-IID skew (0.9232 Acc. on SST-2). The performance of IID federated results, in a quantitative sense, confirms that MTL can generate better results over an isolated centralized training for discrete model performance (e.g., DistilBERT reaching 0.9461 on SST-2). The non-IID metrics highlight that the framework is more robust to data heterogeneity; classification tasks such as SST-2 retain steady performance (e.g., 0.9232 for DistilBERT), but regression tasks such as STS-B show a greater degree of sensitivity. MiniLM is consistently the greatest versatile candidate in both IID and non-IID distributions being able to win 2 of 3 benchmark tasks (QQP and STS-B) with a memory footprint savings of nearly 3× when compared to DistilBERT. This shows that although the framework is robust to categorical benchmarks, the selection of the model needs to be

a delicate bit better, where DistilBERT for sentiment robustness and MiniLM for resource-constrained multi-task stability.

4) Real-Time Inference Analysis: For deployment on edge devices where real-time inference is critical, resource consumption and model footprint are the defining metrics. Figure 2 and Table II highlight that FL reduces individual client GPU memory demands by 77-93% relative to centralized training. TinyBERT and BERT-Mini emerge as the leading candidates for extremely resource-constrained devices (< 0.1 GB and < 0.2 GB per client respectively), albeit with bounded performance on complex tasks. MiniLM and DistilBERT provide a superior performance-efficiency balance for most real-time scenarios, maintaining high accuracy while keeping per-client resource usage significantly lower than the centralized baseline.

5) Communication Efficiency: Communication overhead is the primary bottleneck in federated MTL, with wall-clock training times increasing by 2.2-3.7$\times$ compared to centralized training (Table III). However, as illustrated in Figure 3, the integration of PEFT via LoRA [22] substantially mitigates this cost. LoRA reduces federated training time by 38-62% for most models by transmitting only low-rank adapter weights instead of full model parameters. This optimization enables the framework to maintain competitive accuracy while drastically reducing the bandwidth requirements, making it suitable for bandwidth-limited edge deployments.

V. RELATED WORK

The objective of this section is to survey the pioneering research and modern works on the topic with FL in NLP, MTL, and SLM. We map where these fields intersect to allow efficient distributed learning, and the research gaps we propose a real-time framework, specifically.

A. FL for NLP

Examples of FL in NLP/NLU applications include text classification and personalized language services where raw text cannot be aggregated [13], [15]. Fundamental optimization algorithms-such as FedAvg [13] which defined the client-update-server-aggregation loop-had done so, and FedProx [15] performed regularization to enable enhanced behavior to counter client drift. Further improvements, such as SCAFFOLD [27], mitigate gradient heterogeneity to guarantee convergence stability. Federated MTL methods like MOCHA illustrate that inter-task relatedness can be strengthened under heterogeneous settings through enhanced statistical efficiency and stability to system issues [17]. And newer approaches such as FedBone [18], MIRA [20] and M-Fed [19] take this the step further by separating between shared and task-dependent elements and even specialized encoder-decoder architectures and adapting these into large scale heterogeneous MTL environments. However, several FL-for-NLP studies are still based on very little task diversity or the individual personalization of user at single task.

B. MTL in NLP

MTL for NLP usually uses one of the two paradigms: hard parameter sharing or soft parameter sharing [17], [11]. Unlike many of the popular methods such as adversarial sharing which assumes a task head is separate from each model instance, hard sharing utilizes a common encoder for every model instance, which is useful to save the model from overfitting and cost of training. Unlike hard sharing, soft sharing retains task-specific factors, but promotes similarity in some way through regularization and/or through low interaction. Also, parameter-efficient fine-tuning (PEFT), with adapters [28] and modular shared frameworks such as AdapterHub [29], has allowed rapid multi-task adaptation with less overhead. MTL enhances representation learning by transferring useful linguistic data from a task to another. But this also introduces known risks: task imbalance (over-representation of dominant tasks with lower-level tasks) and negative transfer (one task injuring others). These difficulties are especially high in the federated training case, since the client participants involved in the training process are partially responsible, and the balance of distribution of the data over the rounds is not uniform.

C. SLMs for Efficient NLP

SLMs including DistilBERT [5], MiniLM [7], and ALBERT [30] demonstrate that strict compression and distillation can maintain fair performance at a cost of parameter number in the balance with inference. The common reduction methods are knowledge distillation, parameter sharing, low-rank factorization, and architectural simplification. These resources make those SLMs particularly well suited to edge and distributed learning: transmission overhead is minimized, on-device learning cost on-device is significantly reduced and federated iteration cycles are speeded up. Recent advancements such as EdgeDistilBERT [31] further improve on the state-of-the-art lightweight multi-task FL on edge devices. In multi-task federated systems, SLMs also allow scaling to larger populations of clients and tasks on resource budgets for the system.

D. FL in the presence of heterogeneity in the data

The non-IID data remains one of the worst bottleneck in the FL process. This can lead to biased updates at local

level and cause global optimization failures at global level due to client-dependent language patterns, domain shifts, and label bias. In heterogeneous NLU deployments, existing differences of client tasks (cross-client task diversity) leads to partial representations of client to serve multiple objectives, exacerbating this issue. To mitigate these heterogeneity effects, strategies such as FedBN [32] and low-rank parameterizations like FedPara [33] have been proposed. As in past work, aggregation without considering the structure of a task will reduce velocity and final generalization, and mixing heterogeneity (to aggregate) will degrade speed. Due to the fact convergence velocity may hinder good generalization and final speed depending on task structure, there have been some proposed techniques, like personalized layers, proximal regularization, gradient conflict reducing and encoder-decoder decomposition which bridged this gap [15], [11], [19].

E. Research Gap

Recent systematic reviews [34] have highlighted the importance of practical NLU systems, despite a strong foundation in existing literature.

- No real-time FL frameworks: in the majority of cases, the models are tuned for periodic offline cycles, instead of dynamic adaptation.
- Fractional inclusion of FL + MTL + SLM: the majority of contributions consider efficiency, heterogeneity, and multi-task transfer in isolation.
- Insufficient heterogeneity analysis in NLU FL: the present tests not much considers how task variance, data skew, and communication limitations interact.

This deficiency paves the way for a federated multi-task approach unified in real time and that enables efficient modeling of language but also includes appropriate optimization of distributed NLU with heterogeneity-awareness.

VI. CONCLUSION

Herein we reported a broad study into RT-FedMTL with SLMs on various NLU benchmarks. With our systematic assessment across five architectures, we derived key valuable ideas for implementing decentralized NLU systems. The findings here show the successful balancing of protecting user privacy with a strong optimization of resource usage, with FL achieving up to 93% decrease in per-client GPU resource usage. We discovered a key trade-off in performance and efficiency: for sentiment analysis, DistilBERT delivered consistently the best performance, being notably robust to negative transfer in SST-2. However, for a unified multi-task ensemble, MiniLM proves to be the most flexible and effective architecture and outperformed the larger models in QQP and STS-B tasks, at around 3× lower memory footprint (0.38 GB compared to 1.09 GB). In the case where memory and bandwidth are the bottleneck, architectures such as MiniLM represent the best balance of stability and throughput for real-time federated multi-task scenarios.

For further investigations adaptive task-sharing mechanisms will be explored to tackle negative transfer in multi-head architectures. In addition, we further discuss personalized federated learning techniques and the possibility of differentially private mechanisms implemented in the real-time communication layer, allowing the efficiency gains not to come at the cost of the privacy in the production NLU deployments.

REFERENCES

[1] R. Socher, A. Perelygin, J. Wu, J. Chuang, C. D. Manning, A. Y. Ng, and C. Potts, "Recursive deep models for semantic compositionality over a sentiment treebank," in *Proceedings of the 2013 conference on empirical methods in natural language processing*, 2013, pp. 1631–1642.

[2] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. Bowman, "Glue: A multi-task benchmark and analysis platform for natural language understanding," in *Proceedings of the 2018 EMNLP workshop BlackboxNLP: Analyzing and interpreting neural networks for NLP*, 2018, pp. 353–355.

[3] D. Cer, M. Diab, E. Agirre, I. Lopez-Gazpio, and L. Specia, "Semeval-2017 task 1: Semantic textual similarity multilingual and crosslingual focused evaluation," in *Proceedings of the 11th international workshop on semantic evaluation (SemEval-2017)*, 2017, pp. 1–14.

[4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, 2019, pp. 4171–4186.

[5] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019.

[6] I. Turc, M.-W. Chang, K. Lee, and K. Toutanova, "Well-read students learn better: On the importance of pre-training compact models," *arXiv preprint arXiv:1908.08962*, 2019.

[7] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," *Advances in neural information processing systems*, vol. 33, pp. 5776–5788, 2020.

[8] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated learning: Challenges, methods, and future directions," *IEEE signal processing magazine*, vol. 37, no. 3, pp. 50–60, 2020.

[9] S. Ruder, "An overview of multi-task learning in deep neural networks," *arXiv preprint arXiv:1706.05098*, 2017.

[10] R. Caruana, "Multitask learning," *Machine learning*, vol. 28, no. 1, pp. 41–75, 1997.

[11] T. Yu, S. Kumar, A. Gupta, S. Levine, K. Hausman, and C. Finn, "Gradient surgery for multi-task learning," *Advances in neural information processing systems*, vol. 33, pp. 5824–5836, 2020.

[12] E. Parliament and Council, "Regulation (eu) 2016/679 (general data protection regulation)," *Off J Eur Union*, vol. 119, pp. 1–88, 2016.

[13] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial intelligence and statistics*. Pmlr, 2017, pp. 1273–1282.

[14] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Transactions on Intelligent Systems and Technology (TIST)*, vol. 10, no. 2, pp. 1–19, 2019.

[15] T. Li, A. K. i. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," *Proceedings of Machine learning and systems*, vol. 2, pp. 429–450, 2020.
[16] X. Jiao, Y. Yin, L. Shang, X. Jiang, X. Chen, L. Li, F. Yang, and Q. Liu, "Tinybert: Distilling bert for natural language understanding," in *Findings of the association for computational linguistics: EMNLP 2020*, 2020, pp. 4163–4174.
[17] V. Smith, C.-K. Chiang, M. Sanjabi, and A. S. Talwalkar, "Federated multi-task learning," *Advances in neural information processing systems*, vol. 30, 2017.
[18] Y.-Q. Chen, T. Zhang, X.-L. Jiang, Q. Chen, C.-L. Gao, and W.-L. Huang, "Fedbone: Towards large-scale federated multi-task learning," *Journal of Computer Science and Technology*, vol. 39, no. 5, pp. 1040–1057, 2024.
[19] J. Zhou, W. Bao, J. Wang, D. Zhang, X. Zhang, and Y. Zhang, "Multi-task federated learning with encoder–decoder structure: enabling collaborative learning across different tasks," *International Journal of Machine Learning and Cybernetics*, vol. 16, no. 12, pp. 10 403–10 420, 2025.
[20] A. Elbakary, C. B. Issaid, T. ElBatt, K. Seddik, and M. Bennis, "Mira: A method of federated multi-task learning for large language models," *IEEE Networking Letters*, 2025.
[21] S. Reddi, Z. Charles, M. Zaheer, Z. W. Garrett, K. Rush, J. Konečný, S. Kumar, and H. B. McMahan, "Adaptive federated optimization," *arXiv preprint arXiv:2003.00295*, 2020.
[22] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen *et al.*, "Lora: Low-rank adaptation of large language models." *Iclr*, vol. 1, no. 2, p. 3, 2022.
[23] C. J. Van Rijsbergen, *Information retrieval*. Butterworths, 1979.
[24] L. Sharma, L. Graesser, N. Nangia, and U. Evci, "Natural language understanding with the quora question pairs dataset," *arXiv preprint arXiv:1907.01041*, 2019.
[25] K. Pearson, "Vii. note on regression and inheritance in the case of two parents," *proceedings of the royal society of London*, vol. 58, no. 347-352, pp. 240–242, 1895.
[26] C. Spearman, "The proof and measurement of association between two things." 1961.
[27] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "Scaffold: Stochastic controlled averaging for federated learning," in *International conference on machine learning*. PMLR, 2020, pp. 5132–5143.
[28] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-efficient transfer learning for nlp," in *International conference on machine learning*. PMLR, 2019, pp. 2790–2799.
[29] J. Pfeiffer, A. Rücklé, C. Poth, A. Kamath, I. Vulić, S. Ruder, K. Cho, and I. Gurevych, "Adapterhub: A framework for adapting transformers," in *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*, 2020, pp. 46–54.
[30] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, "Albert: A lite bert for self-supervised learning of language representations," *arXiv preprint arXiv:1909.11942*, 2019.
[31] D. Gholamiangonabadi, "Federated learning for natural language processing," Ph.D. dissertation, University of Western Ontario, 2025.
[32] X. Li, M. Jiang, X. Zhang, M. Kamp, and Q. Dou, "Fedbn: Federated learning on non-iid features via local batch normalization," *arXiv preprint arXiv:2102.07623*, 2021.
[33] H.-W. Nam, M. Ye-Bin, and T.-H. Oh, "Fedpara: Low-rank hadamard product for communication-efficient federated learning," *arXiv preprint arXiv:2108.06098*, 2021.
[34] Y. Khan, D. Sánchez, and J. Domingo-Ferrer, "Federated learning-based natural language processing: a systematic literature review," *Artificial Intelligence Review*, vol. 57, no. 12, p. 320, 2024.

Quang-Vinh Pham received his B.Eng. degree in Electronics and Telecommunications and his Master's degree in Data Science from Quy Nhon University, Vietnam. He is currently an AI Engineer at TMA Tech Group, Vietnam. His research interests include Natural Language Processing, Computer Vision, and Physical AI.

Quang-Hung Le received the Ph.D. degree in Computer Science from the University of Engineering and Technology, Vietnam National University, Hanoi, in 2016. He is currently a lecturer with the Department of Information Technology, Quy Nhon University, Vietnam. His research interests include Natural Language Processing, Machine Learning, Artificial Intelligence, and Recommender Systems.